



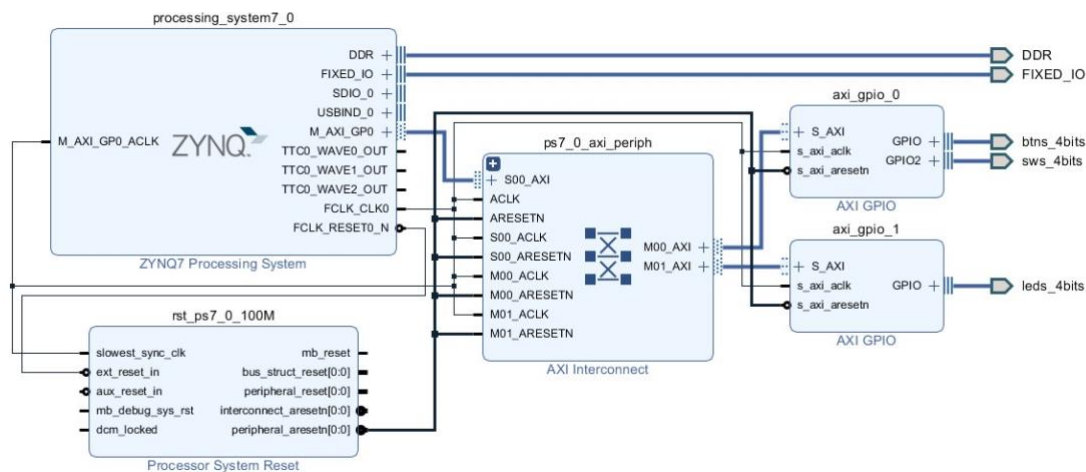
ECE3623 Embedded System Design Laboratory

Robert A. Esposito, PhD
resposit@temple.edu



FreeRTOS Data Queue on Zybo v2

In this Lab you will investigate the implementation of a FreeRTOS data *queue* on the Zybo board. In Lab 1, the operating system was multitasking with FreeRTOS but did not pass data between the tasks. Lab 3 here will use the FreeRTOS *queue* function to pass data.



Complete the Vivado hardware design above (same as Lab 1) and launch SDK. In SDK do *File...New...Application Project* then enter a new project name and select *freertos10_xilinx* as the OS Platform. Not *Finish* but *Next* to continue. In the *New Project...Templates* select *an empty template to use*. Then copy and paste the shell code at the end of the document in to the template.

1. Create a FreeRTOS **main()** application that creates two tasks and a queue. The queue can hold 4 entries. The FreeRTOS functions *XQueueCreate()*, *XQueueReceive()* and *XQueueSend()* utilizes a handle *xQueueBtnSw*.
2. Create a FreeRTOS task **TaskQueueSend** that sends a 1 to the Queue every time BTN0 is pressed and illuminates the LEDS according to the data in the queue (i.e. if the queue is empty then the LEDS illuminate 0000, and as entries are added to the queue, the LEDS start to light up in sequence 0001, 0011, 0111, 1111). SW0 resets the queue to empty.

3. Create a FreeRTOS task ***TaskQueueReceive*** that receives a value from the Queue every time BTN1 is pressed and illuminates the LEDS according to the data in the queue (i.e. if the queue is full then the LEDS illuminate 1111, and as entries are received from the queue, the LEDS start to light up in sequence 0111, 0011, 0001, 0000).

FALL 2023

```
//LAB 3 Shell Code
```

```
// includes
```

```
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "xil_printf.h"
#include "xparameters.h"
#include "xgpio.h"
#include "xstatus.h"
```

```
// defines
```

```
#define BTNSW_DEVICE_ID    XPAR_AXI_GPIO_0_DEVICE_ID
#define LED_DEVICE_ID      XPAR_AXI_GPIO_1_DEVICE_ID
#define BTN_CHANNEL 1
#define SW_CHANNEL 2
#define LED_CHANNEL 1
```

```
// global variables
```

```
static TaskHandle_t xTaskQueueSend; // FreeRTOS task handle static
TaskHandle_t xTaskQueueReceive; // FreeRTOS task handle static
QueueHandle_t xQueueBtnSw; // FreeRTOS queue handle
```

```
XGpio LedInst; // GPIO device driver instance
XGpio BtnSwInst; // GPIO device driver instance
```

```
// function declarations int
initialize();
static void vTaskQueueSend(void *pvParameters); static
void vTaskQueueReceive(void *pvParameters);
```

```
// *****MAIN ***** int
main(void) {
```

```
// Run Initialization
if (initialize() != XST_SUCCESS) {
xil_printf("GPIO setup failed!\r\n");
}
```

```
//Create Tasks and Start Scheduler
```

```
}
```

```
// *****Initialize***** int
initialize() {
int status;
```

```
// Initialize LEDs (GPIO driver initialization) status =
XGpio_Initialize(&LedInst, LED_DEVICE_ID); if
(status != XST_SUCCESS)
return XST_FAILURE;
```

```
// Initialize Push Buttons and Switches (GPIO driver initialization)
status = XGpio_Initialize(&BtnSwInst, BTNSW_DEVICE_ID); if
(status != XST_SUCCESS)
return XST_FAILURE;
```

```
// Set LED directions to output
```

```
// Set button directions to input
```

```
// Set switch directions to input
```

```
}
```

```
// *****TaskQueueSend*****
```

```
static void vTaskQueueSend(void *pvParameters) {
```

```
while (1) {
```

```
//Send Logic
```

```
}
```

```
}
```

```
// *****TaskQueueReceive***** static
```

```
void vTaskQueueReceive(void *pvParameters) {
```

```
while (1) {
```

```
//Receive Logic
```

```
}
```

```
}
```